Sesión 11

NoSQL y Cloud

Carlos César Sánchez Coronel

BASES DE DATOS

2026

Contenido general

1. Introducción a NoSQL
2. MongoDB
3. Redis
4. Neo4j
5. Cassandra
6. Elasticsearch
7. Comparativa SQL vs NoSQL
8. Integración con IA
9. Infraestructura Cloud para NoSQL
10. Ejercicios Resueltos
11. Glosario

Contenido de la sección

1. Introducción a NoSQL
2. MongoDB
3. Redis
4. Neo4j
5. Cassandra
6. Elasticsearch
7. Comparativa SQL vs NoSQL
8. Integración con IA
9. Infraestructura Cloud para NoSQL
10. Ejercicios Resueltos
11. Glosario

¿Por qué NoSQL?

- Escalabilidad vertical costosa (más CPU/RAM en un solo servidor).
- Esquema rígido (schema-on-write) dificulta cambios frecuentes.
- Dificultad para manejar grandes volúmenes de datos no estructurados.
- Rendimiento limitado en operaciones de lectura/escritura masivas.

Teorema CAP

En sistemas distribuidos no se pueden garantizar simultáneamente:

- **C**onsistencia: todos ven los mismos datos.
- **D**isponibilidad: siempre responde.
- **T**olerancia a Particiones: funciona pese a fallos de red.

CP HBase, MongoDB (configuración típica)

AP Cassandra, DynamoDB, CouchDB

CA Sistemas tradicionales (no toleran particiones)

Tipos de Bases de Datos NoSQL

Documentales	MongoDB, Couchbase
Clave-Valor	Redis, DynamoDB
Grafos	Neo4j, Amazon Neptune
Columnares	Cassandra, HBase
Motores de búsqueda	Elasticsearch, Solr

Contenido de la sección

1. Introducción a NoSQL
2. MongoDB
3. Redis
4. Neo4j
5. Cassandra
6. Elasticsearch
7. Comparativa SQL vs NoSQL
8. Integración con IA
9. Infraestructura Cloud para NoSQL
10. Ejercicios Resueltos
11. Glosario

Modelo de Datos en MongoDB

- Almacena documentos en formato BSON (JSON binario).
- Esquema flexible: diferentes documentos pueden tener distintos campos.
- Soporta arrays, subdocumentos y tipos de datos avanzados.

Ejemplo de documento

```
{
  "_id": ObjectId("507f1f77bcf86cd799439011"),
  "nombre": "Ana Pérez",
  "email": "ana@mail.com",
  "direccion": {
    "calle": "Av. Siempre Viva",
    "ciudad": "Madrid"
  },
  "pedidos": [
    { "fecha": "2025-01-10", "total": 150 },
    { "fecha": "2025-02-15", "total": 200 }
  ]
}
```

Consultas con PyMongo

Operaciones básicas

```python
from pymongo import MongoClient

client = MongoClient('localhost', 27017)
db = client['tienda']
coleccion = db['clientes']

# Insertar
coleccion.insert_one({"nombre": "Luis", "email": "luis@mail.com"})

# Buscar
result = coleccion.find({"ciudad": "Madrid"})
for doc in result:
    print(doc)

# Actualizar
coleccion.update_one({"nombre": "Luis"}, {"$set": {"ciudad": "Barcelona"}})

# Eliminar
coleccion.delete_one({"nombre": "Luis"})
```

Índices y Escalabilidad

- **Índices**: simples, compuestos, geoespaciales, texto.
- **Sharding**: particionamiento horizontal por clave de shard.
- **Réplicas**: alta disponibilidad (replica sets).

Contenido de la sección

1. Introducción a NoSQL
2. MongoDB
3. Redis
4. Neo4j
5. Cassandra
6. Elasticsearch
7. Comparativa SQL vs NoSQL
8. Integración con IA
9. Infraestructura Cloud para NoSQL
10. Ejercicios Resueltos
11. Glosario

Características de Redis

- Base de datos en memoria (in-memory).
- Latencia de microsegundos.
- Estructuras ricas: strings, hashes, listas, sets, sorted sets, etc.
- Operaciones atómicas (INCR, LPUSH, ZADD).

Conexión desde Python (redis-py)

```python
import redis

r = redis.Redis(host='localhost', port=6379, decode_responses=True)

# Strings
r.set('clave', 'valor')
print(r.get('clave'))

# Contadores
r.incr('contador')

# Listas
r.lpush('lista', 'elemento1', 'elemento2')
elementos = r.lrange('lista', 0, -1)

# Sorted sets (para ranking)
r.zadd('ranking', {'A': 1, 'B': 2})
ranking = r.zrevrange('ranking', 0, -1, withscores=True)
```

Persistencia y Escalabilidad

- **Persistencia**: RDB (snapshots) y AOF (log de operaciones).
- **Replicación**: maestro-esclavo.
- **Redis Cluster**: sharding automático.

Contenido de la sección

1. Introducción a NoSQL
2. MongoDB
3. Redis
4. Neo4j
5. Cassandra
6. Elasticsearch
7. Comparativa SQL vs NoSQL
8. Integración con IA
9. Infraestructura Cloud para NoSQL
10. Ejercicios Resueltos
11. Glosario

Modelo de Grafos

- **Nodos**: entidades (Persona, Empresa).
- **Relaciones**: conexiones dirigidas con tipo y propiedades.
- **Propiedades**: atributos de nodos y relaciones.

Consultas con Cypher

Crear nodos y relaciones

```
CREATE (a:Persona {nombre: 'Ana', edad: 30})
CREATE (b:Persona {nombre: 'Luis', edad: 32})
CREATE (a)-[:AMIGO_DE]->(b)
```

Consultar amigos de Ana

```
MATCH (p:Persona {nombre: 'Ana'})-[:AMIGO_DE]->(amigo)
RETURN amigo.nombre
```

Integración con Python (neo4j driver)

```python
from neo4j import GraphDatabase

driver = GraphDatabase.driver("bolt://localhost:7687", auth=("neo4j", "password"))

def get_friends(tx, nombre):
    query = "MATCH (p:Persona {nombre: $nombre})-[:AMIGO_DE]->(amigo) RETURN amigo.nombre"
    result = tx.run(query, nombre=nombre)
    return [record["amigo.nombre"] for record in result]

with driver.session() as session:
    amigos = session.execute_read(get_friends, "Ana")
    print(amigos)

driver.close()
```

Contenido de la sección

1. Introducción a NoSQL
2. MongoDB
3. Redis
4. Neo4j
5. Cassandra
6. Elasticsearch
7. Comparativa SQL vs NoSQL
8. Integración con IA
9. Infraestructura Cloud para NoSQL
10. Ejercicios Resueltos
11. Glosario

Arquitectura y Modelo de Datos

- Peer-to-peer, sin maestro.
- Escalabilidad lineal.
- Clave primaria compuesta: **clave de partición + columnas de clustering**.

Tabla para mensajes por usuario

```
CREATE TABLE mensajes (
    usuario_id UUID,
    timestamp TIMESTAMP,
    mensaje TEXT,
    PRIMARY KEY ((usuario_id), timestamp)
) WITH CLUSTERING ORDER BY (timestamp DESC);
```

Conexión desde Python (cassandra-driver)

```python
from cassandra.cluster import Cluster

cluster = Cluster(['127.0.0.1'])
session = cluster.connect('mi_keyspace')

rows = session.execute("SELECT * FROM mensajes WHERE usuario_id = %s", [usuario_id])
for row in rows:
    print(row.timestamp, row.mensaje)

cluster.shutdown()
```

Contenido de la sección

1. Introducción a NoSQL
2. MongoDB
3. Redis
4. Neo4j
5. Cassandra
6. Elasticsearch
7. Comparativa SQL vs NoSQL
8. Integración con IA
9. Infraestructura Cloud para NoSQL
10. Ejercicios Resueltos
11. Glosario

Conceptos Clave

- **Índice**: colección de documentos.
- **Documento**: unidad de información (JSON).
- **Shard**: partición de un índice.
- **Inverted index**: estructura para búsqueda rápida de términos.

Consultas con Python (elasticsearch-py)

```python
from elasticsearch import Elasticsearch
from datetime import datetime

es = Elasticsearch(['localhost:9200'])

# Indexar documento
doc = {
    'timestamp': datetime.now(),
    'nivel': 'ERROR',
    'mensaje': 'Timeout en conexión'
}
es.index(index='logs', body=doc)

# Buscar
query = {
    'query': {
        'match': {'mensaje': 'Timeout'}
    }
}
resp = es.search(index='logs', body=query)
for hit in resp['hits']['hits']:
    print(hit['_source'])
```

Contenido de la sección

1. Introducción a NoSQL
2. MongoDB
3. Redis
4. Neo4j
5. Cassandra
6. Elasticsearch
7. Comparativa SQL vs NoSQL
8. Integración con IA
9. Infraestructura Cloud para NoSQL
10. Ejercicios Resueltos
11. Glosario

Comparativa general

Característica	SQL	NoSQL
Esquema	Fijo (schema-on-write)	Flexible (schema-on-read)
Transacciones	ACID	BASE (consistencia eventual)
Escalabilidad	Vertical	Horizontal
Consultas	Estructuradas (JOINs)	Según tipo (no joins)
Casos de uso	Transaccionales	Big Data, tiempo real

Comparativa entre tipos NoSQL

Tipo	Ventajas	Casos típicos
Documental	Flexible, fácil	Catálogos, CMS
Clave-Valor	Ultra rápido	Caché, sesiones
Grafos	Relaciones complejas	Recomendación, fraude
Columnar	Alta escalabilidad	IoT, series temporales
Búsqueda	Texto completo	Logs, búsqueda web

Contenido de la sección

1. Introducción a NoSQL
2. MongoDB
3. Redis
4. Neo4j
5. Cassandra
6. Elasticsearch
7. Comparativa SQL vs NoSQL
8. Integración con IA
9. Infraestructura Cloud para NoSQL
10. Ejercicios Resueltos
11. Glosario

Almacenamiento de Embeddings

- En MongoDB se guardan como arrays y se usa `$cosineSimilarity`.
- En Elasticsearch con campo `dense_vector`.
- Ejemplo con MongoDB:

```
embedding = [0.12, 0.45, ...]  # vector de 300 dimensiones
coleccion.insert_one({'texto': 'ejemplo', 'embedding': embedding})
```

Caché de Modelos y Resultados con Redis

```python
import redis, pickle
r = redis.Redis()

# Cachear modelo serializado
modelo_bytes = pickle.dumps(modelo_entrenado)
r.set('modelo:recomendacion', modelo_bytes)

# Cachear predicción con TTL
r.setex('pred:user:123', 3600, pickle.dumps(prediccion))
```

Sesiones de Usuario en Chatbots (Redis)

```python
# Guardar contexto de conversación
import json
r.setex(f'session:{session_id}', 1800, json.dumps({
    'ultimo_intento': 'hola',
    'historial': ['mensaje1', 'mensaje2']
}))
```

Recomendaciones con Grafos (Neo4j)

Recomendar productos basados en usuarios similares

```
MATCH (u:Usuario {id:123})-[:COMPRO]->(p:Producto)<-[:COMPRO]-(otros:Usuario)
MATCH (otros)-[:COMPRO]->(recom:Producto)
WHERE NOT (u)-[:COMPRO]->(recom)
RETURN recom, COUNT(*) AS frecuencia
ORDER BY frecuencia DESC
LIMIT 10
```

Contenido de la sección

1. Introducción a NoSQL
2. MongoDB
3. Redis
4. Neo4j
5. Cassandra
6. Elasticsearch
7. Comparativa SQL vs NoSQL
8. Integración con IA
9. Infraestructura Cloud para NoSQL
10. Ejercicios Resueltos
11. Glosario

Servicios gestionados en AWS

DynamoDB	Clave-valor/documental serverless.
DocumentDB	Compatible con MongoDB.
ElastiCache	Redis/Memcached gestionado.
Neptune	Base de datos de grafos.
OpenSearch	Sucesor de Elasticsearch.

Servicios gestionados en Azure

Cosmos DB Multimodelo (documental, clave-valor, columna, grafo).

Table Storage Clave-valor.

Cache for Redis Redis gestionado.

Servicios gestionados en GCP

Firestore	Documental.
Bigtable	Columna ancha (similar a HBase).
Memorystore	Redis.

Dimensionamiento y Costos (Ejemplo DynamoDB)

- **RCU/WCU**: Unidades de capacidad de lectura/escritura.
- Ejemplo: 1 millón escrituras/mes = $1.25, 10 millones lecturas/mes = $2.50, 100 GB almacenamiento = $25.
- Total aproximado: $28.75/mes.

Contenido de la sección

1. Introducción a NoSQL
2. MongoDB
3. Redis
4. Neo4j
5. Cassandra
6. Elasticsearch
7. Comparativa SQL vs NoSQL
8. Integración con IA
9. Infraestructura Cloud para NoSQL
10. Ejercicios Resueltos
11. Glosario

Ejercicio 1: Catálogo de productos en MongoDB

Enunciado: Modelar un documento para productos con nombre, precio, categoría, etiquetas y especificaciones variables.

Solución

```python
from pymongo import MongoClient

client = MongoClient()
db = client['tienda']
productos = db['productos']

producto1 = {
    'nombre': 'Laptop Gamer',
    'precio': 1200,
    'categoria': 'Electrónica',
    'etiquetas': ['laptop', 'gaming'],
    'especificaciones': {'ram': '16GB', 'procesador': 'i7'}
}
producto2 = {
    'nombre': 'Mouse',
    'precio': 25,
    'categoria': 'Electrónica',
    'etiquetas': ['mouse', 'inalámbrico'],
    'especificaciones': {'tipo': 'óptico'}
}
productos.insert_many([producto1, producto2])
```

Ejercicio 2: Sistema de votación con Redis

Enunciado: Usar Redis para contador de votos y ranking con sorted set.

Solución

```python
import redis

r = redis.Redis()

# Votar
r.incr('votos:candidato:A')
r.incr('votos:candidato:B')
r.incr('votos:candidato:A')

# Ranking
r.zincrby('ranking', 1, 'A')
r.zincrby('ranking', 2, 'B')
r.zincrby('ranking', 1, 'A')

print(r.get('votos:candidato:A'))
print(r.zrevrange('ranking', 0, -1, withscores=True))
```

Ejercicio 3: Detección de fraude con grafos (Neo4j)

Enunciado: Detectar ciclos de transferencias rápidas entre cuentas.

Solución

```
-- Crear cuentas y transferencias
CREATE (a:Cuenta {id: 'A'}), (b:Cuenta {id: 'B'}), (c:Cuenta {id: 'C'})
CREATE (a)-[:TRANSFIERE {monto: 1000, fecha: '2025-03-01'}]->(b)
CREATE (b)-[:TRANSFIERE {monto: 900, fecha: '2025-03-02'}]->(c)
CREATE (c)-[:TRANSFIERE {monto: 800, fecha: '2025-03-03'}]->(a)

-- Buscar ciclos de longitud 3
MATCH p = (a)-[:TRANSFIERE*3]-(a)
RETURN p
```

Ejercicio 4: Logs en Elasticsearch

Enunciado: Indexar logs y buscar errores de los últimos 5 minutos.

Solución

```python
from elasticsearch import Elasticsearch
from datetime import datetime, timedelta

es = Elasticsearch()

doc = {
    'timestamp': datetime.now(),
    'nivel': 'ERROR',
    'mensaje': 'Timeout en conexión'
}
es.index(index='logs', body=doc)

hace_5min = datetime.now() - timedelta(minutes=5)
query = {
    'query': {
        'bool': {
            'must': [
                {'match': {'nivel': 'ERROR'}},
                {'range': {'timestamp': {'gte': hace_5min}}}
            ]
        }
    }
}
```

Contenido de la sección

1. Introducción a NoSQL
2. MongoDB
3. Redis
4. Neo4j
5. Cassandra
6. Elasticsearch
7. Comparativa SQL vs NoSQL
8. Integración con IA
9. Infraestructura Cloud para NoSQL
10. Ejercicios Resueltos
11. Glosario

Términos Clave

NoSQL	Not Only SQL, bases de datos no relacionales.
Teorema CAP	Consistencia, Disponibilidad, Tolerancia a Particiones.
MongoDB	Base de datos documental.
Redis	Base de datos en memoria clave-valor.
Neo4j	Base de datos de grafos.
Cassandra	Base de datos columnar distribuida.
Elasticsearch	Motor de búsqueda y analítica.
DynamoDB	NoSQL serverless de AWS.
Cosmos DB	Base de datos multimodelo de Azure.
Sharding	Particionamiento horizontal.
Replicación	Copia de datos en varios nodos.
Embedding	Representación vectorial para IA.

¿Preguntas?

¡Gracias por su atención!